Concurrency and Synchronization Roadmap

01 / Mutual Exclusion (Today)

Define the core problem of race conditions in concurrent execution and explore foundational software-based solutions to enforce mutual exclusion.

02 / Hardware-Based Solutions

Examine low-level hardware mechanisms including interrupt disabling and atomic instructions like Test-and-Set (TS) for synchronization.

03 / Semaphores

Master the OS-provided synchronization primitive used for signaling, counting, and managing access to shared resources.

04 / Classic Synchronization Problems

Analyze canonical problems such as Producer-Consumer, Readers-Writers, and Dining Philosophers to understand practical synchronization challenges.

05 / Deadlock, Starvation & Livelock

Identify system failures caused by synchronization issues: deadlock (circular wait), starvation (indefinite postponement), and livelock.

Concurrency and Synchronization Roadmap

- Concurrency and Synchronization
  - ➢ Mutual exclusion and critical section
  - ➢ Lost update problem and busy waiting vs. blocking
  - ➢ Interrupt disabling and spinlock
  - ➢ Semaphores and Readers/Writers problem
  - ➢ Deadlock and starvation

Operating Systems

Mutual Exclusion and Critical Section

Chapter 5 — Concurrency: Mutual Exclusion and Synchronization

Key Terms in Concurrency

Atomic Operation

Critical Section

Deadlock

Livelock

Mutual Exclusion

Race Condition

Starvation

See also Table 5.1 in the textbook.

Mutual Exclusion and Critical Section

Foundations of Concurrency & Synchronization

Critical Section Problem

The core challenge where multiple processes compete for shared resources. Without control, race conditions and data inconsistency occur.

Mutual Exclusion Rule

A fundamental requirement: only one process can execute its critical section at any time, ensuring no two processes modify shared data concurrently.

System Design Goals

Ensure three key properties: mutual exclusion, progress (no deadlock), and bounded waiting (fairness) for efficient resource access.

THINK: What is Concurrency?

01 Core Concept

Concurrency enables multiple threads or processes to execute in an **interleaved** fashion on a CPU. Context switches can occur between any two machine instructions, creating the illusion of parallelism even on a single-core processor.

02 Key Advantages

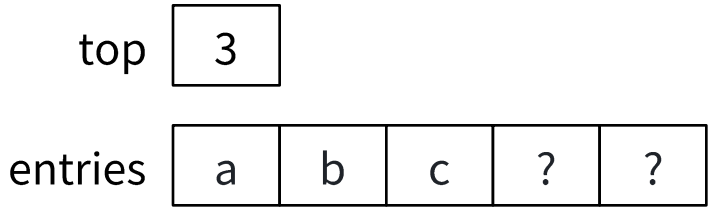
✅ **Higher CPU Utilization:** Overlaps I/O waiting time with computation to keep the processor busy.
✅ **Responsive Systems:** Ensures interactive applications (e.g., GUIs) remain responsive during heavy background tasks.

03 Critical Risks

❗ **Shared State Hazard:** Bugs (e.g., race conditions) arise **only** when threads access shared memory/variables.
✅ **Safe Isolation:** Thread-local (private) variables are completely immune to concurrency issues.

Key Takeaway: Concurrency is a powerful tool for efficiency, but its complexity stems entirely from managing shared mutable state.

A Concurrent Update That Goes Wrong



Thread A

a1 entries[top] = 'x';
a2 top++;

Thread B

b1 entries[top] = 'y';
b2 top++;

Execution order: **a1, b1, a2, b2**

Result: top = 5, entries = [a, b, c, y, ?]

The value 'x' written by thread A is overwritten and lost. This is the **lost-update problem**, caused by a **race condition**.

Race Condition: The Problem

A **Race Condition** occurs when the final result of concurrent operations depends on the **unpredictable interleaving** of thread execution. This non-deterministic behavior leads to inconsistent and hard-to-reproduce bugs.

01. Case Study: count++ is not an atomic operation

The simple increment operation is actually three discrete steps:
1. **LOAD** count → register (Read current value from memory) 2. **ADD** register + 1 (Increment the value in register) 3. **STORE** register → count (Write back)

Sequential Execution (Correct)

Thread A completes all steps (Load → Add → Store) before Thread B starts.
Final Result: count = 2

Interleaved Execution (Incorrect)

Thread B loads the same value before A stores, leading to a **Lost Update**.
Final Result: count = 1

Critical Section: Where the Problem Occurs

DEFINITION: A **Critical Section (CS)** is a segment of code that accesses **shared resources** (e.g., variables, files, hardware devices). It is the precise point in a program where uncontrolled concurrent access leads to instability.

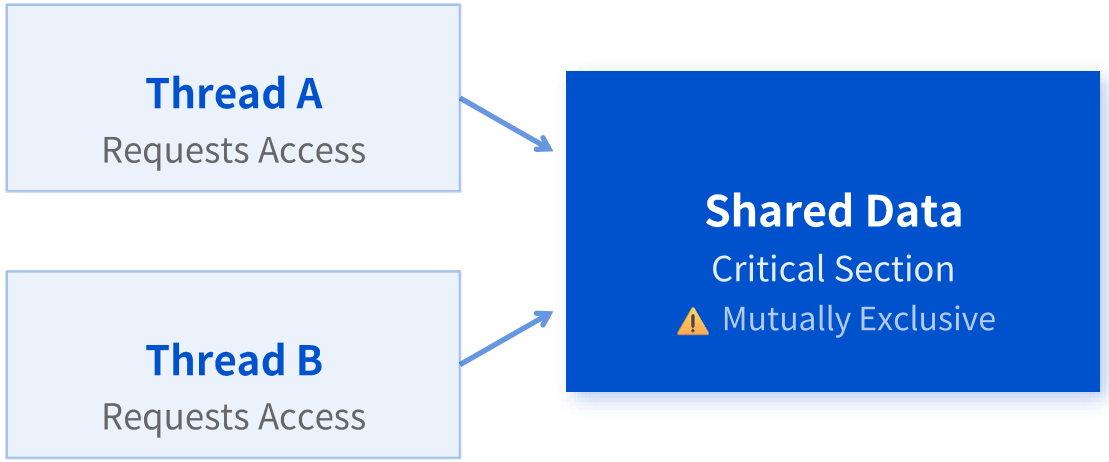
01. Race Condition Trigger

If multiple threads execute their CS simultaneously, a race condition occurs, leading to inconsistent data states and unpredictable program behavior.

02. Safe Remainder Section

Code outside the CS (the Remainder Section) has no shared resource access. This part is inherently thread-safe and can be executed in parallel without synchronization.

Concurrency Access Model



Marking the Critical Section

The programmer marks critical sections with special code, referred to generically as **EnterCS()** and **LeaveCS()**. They must guarantee that at most one process or thread runs inside the critical section.

Thread A

```
...
EnterCS();
entries[top] = 'x';
top++;
LeaveCS();
...
```

Thread B

```
...
EnterCS();
entries[top] = 'y';
top++;
LeaveCS();
...
```

The rest of this chapter studies how to implement EnterCS() and LeaveCS().

Three Requirements for a Correct Solution

01. Mutual Exclusion

At most **one** process can execute inside the Critical Section (CS) at any time. No two processes may be in the CS simultaneously.

Core Goal: Prevent race conditions by ensuring strict isolation.

02. Progress

If no process is in the CS, the system must select a waiting process to enter. Decision cannot be postponed indefinitely.

Core Goal: Prevent group deadlock and ensure system liveness.

03. Bounded Waiting

There exists a limit on the number of times other processes can enter the CS after a request is made, before it is granted.

Core Goal: Eliminate starvation and guarantee fairness.

"These three conditions are the cornerstone of any correct synchronization algorithm."

General Thread Structure (4-Part Model)

01 Entry Section

Requests permission to enter the Critical Section. This acts as the initial gatekeeping step for thread synchronization.

02 Critical Section

The core segment accessing shared resources. Only one thread may execute here at a time to prevent race conditions.

03 Exit Section

Releases the acquired synchronization lock, signaling that the Critical Section is now available for other waiting threads.

04 Remainder Section

Contains thread-specific private tasks and computations that do not interact with shared resources, requiring no synchronization.

Cyclic Execution Logic: The process forms a continuous loop: After completing the Remainder Section, the thread typically returns to the Entry Section to attempt access again, creating an infinite cycle of concurrent execution.

What Is a Critical Section?

A **critical section** is a section of code within a process that requires access to shared resources, and which may **not** be executed while another process is in a corresponding section of code.

In the stack example, the critical section is the two-instruction block:
entries[top] = 'x'; top++; (a1–a2 in thread A, b1–b2 in thread B)
Any instruction sequence that reads and updates a shared variable non-atomically is a candidate critical section.

Structure of Critical Section

Every process that needs to access shared resources follows a specific structure:

```
do {
    entry_section();        // Request permission to enter

    critical_section();     // Access shared resources

    exit_section();         // Release access permission

    remainder_section();    // Non-critical code
} while (true);
```

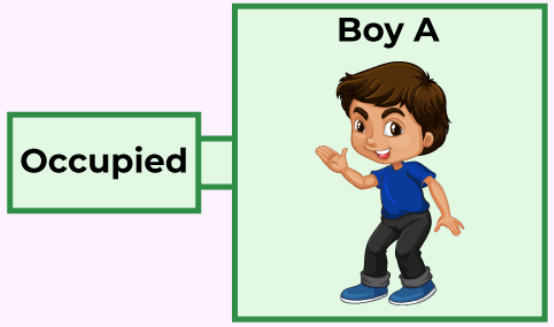
- **Entry Section**: Code that requests permission to enter the critical section
- **Critical Section:** Code segment accessing shared resources
- **Exit Section**: Code that releases the permission
- **Remainder Section**: Rest of the process code

Mutual Exclusion

The execution of code in critical sections must maintain **mutual exclusion**: *only one thread or process is allowed to be in the critical section at a time.*

Put another way: if thread A is inside its critical section, then every other thread must not be allowed to enter its corresponding critical section.
When thread A leaves, only then may a waiting thread be chosen to enter.

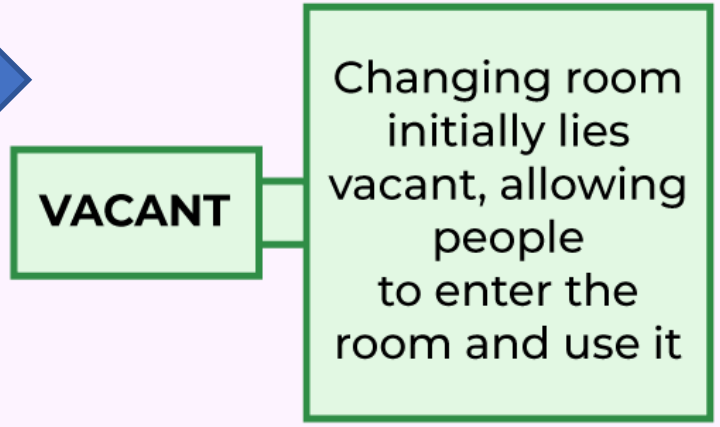
“In the clothes section of a supermarket, two people are shopping for clothes.”

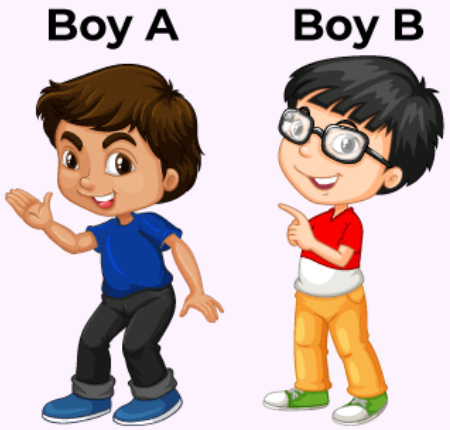


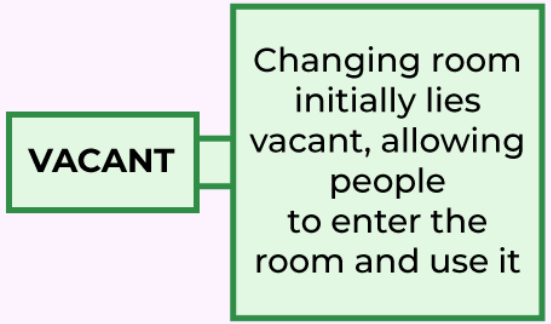
Since Boy A is Inside the changing room, the sign on it prevent the other from entering the room.



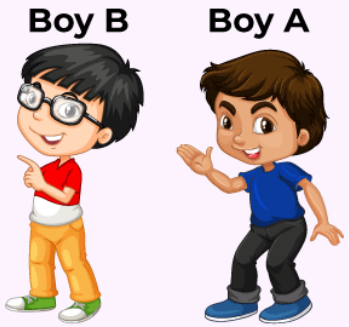
Boy B has to wait outside the changing room till boy A comes out







Since Boy A is done using the changing room, he vacates it- making the sign outside the room change.



Since the changing room has been vacted by Boy A, Boy B can enter the hanging room.

Conclusion

In conclusion, mutual exclusion is a key concept in synchronization that ensures only one process accesses a shared resource at a time. This prevents conflicts and data corruption, making sure that processes run smoothly and correctly. *By using mutual exclusion mechanisms, we can create stable and reliable systems that handle multiple processes efficiently.*

Dekker's Algorithm

The First Correct Software Solution for Mutual Exclusion

01 Core Intuition

Relies on two shared variables to mediate access without hardware locks: • **flag[i]**: Boolean indicating process i’ s intent to enter the critical section. • **turn**: Integer (0 or 1) that resolves conflicts by assigning priority.

02 Real-World Analogy

Imagine two people wanting to use a single restroom: 1. **Signal:** Raise your hand (flag[i] = true) to show you want in. 2. **Check:** If the other person also raised their hand, check the "turn" sign. 3. **Yield:** If it’ s not your turn, lower your hand and wait for your turn.

03 Pseudocode (Process P0)

```
// Shared global variables
bool flag[2];    // flag[i] = true means Pi wants to enter CS
int turn;        // indicates whose turn to enter

// Process P0
flag[0] = true;
while (flag[1]) {
    if (turn == 1) {
        flag[0] = false;     // P0 temporarily withdraw request
        while (turn == 1);   // spin wait until turn switches
        flag[0] = true;      // re-claim intent
    }
}
// -------- Critical Section --------
turn = 1;                    // hand turn over to P1
flag[0] = false;             // clear request
```

Pros

The first rigorously proven correct software-only solution for two processes. No specialized hardware instructions required.

Cons

Complex nested busy-wait loops make it hard to understand and verify. Extending to N processes is impractical and error-prone.

Exercises




In Dekker's algorithm for two processes, if both processes set flag1 = true and flag2 = true at the same time and turn = 1, Process 1 must drop its flag and wait for Process 2 to enter the critical section.



Dekker's algorithm satisfies the three requirements for critical section problems: mutual exclusion, progress and bounded waiting.
(1) Briefly explain what mutual exclusion means in this two-process scenario.
(2) What is the main disadvantage of Dekker's algorithm?

Dekker's algorithm uses busy waiting. The waiting process repeatedly checks shared variables and consumes CPU cycles without doing useful work.



```
flag1 = false;
turn = 1;

// Entry section
flag1 = true;
while (flag2) {
    if (turn == 2) {
        flag1 = false;
        while (turn == 2) {}
        flag1 = true;
    }
}
// Critical Section
// ...
// Exit section
turn = 2;
flag1 = false;
```

Question:
Suppose flag1 = true, flag2 = true, and turn = 1.
Describe the execution sequence of Process 1 and Process 2, and state which process enters the critical section. Explain why.

Peterson's Algorithm

Simplified & Elegant Mutual Exclusion

Core Intuition

Utilizes two shared variables: **flag[i]** signals a process' s intent to enter the critical section, while **turn** acts as a "politeness" variable to resolve conflicts by voluntarily yielding priority.

Analogy: Two People at a Door

1. **Signal:** Raise your hand (*flag[i] = true*) to show you want to enter.
2. **Yield:** Politely say "After you" (*turn = other*) to the other person.
3. **Wait:** Only pause if the other person also raised their hand AND it' s their turn.

Pseudocode for Process P0

```
flag[0] = true;        // Declare intent to enter
turn = 1;              // Yield priority to P1
while (flag[1] && turn == 1); // Wait condition
// --- Critical Section Execution ---
flag[0] = false;       // Exit the critical section
```

Key Advantages

Minimalist code structure, logically elegant, and satisfies all mutual exclusion requirements. It is mathematically provable for correctness.

Limitations

Restricted to exactly two processes only. It uses busy waiting (CPU polling), which is inefficient for scenarios with long wait times.

Peterson's Algorithm is a classic software-based solution for the critical section problem in operating systems. It ensures mutual exclusion between two processes, meaning only one process can access a shared resource at a time, thus preventing race conditions.

The algorithm uses two shared variables:

flag[i]: shows whether process i wants to enter the critical section.
turn: indicates whose turn it is to enter if both processes want to access the critical section at the same time.

The Algorithm

For process Pi:

```
do {
    flag[i] = true;        // Pi wants to enter
   turn = j;               // Give turn to Pj

    while (flag[j] && turn == j);  // Wait if Pj also wants to enter

    // Critical Section

    flag[i] = false;       // Pi leaves critical section

    // Remainder Section
} while (true);
```

For process Pj:

```
do {
    flag[j] = true;
    turn = i;

    while (flag[i] && turn == i);

    // Critical Section

    flag[j] = false;

    // Remainder Section
} while (true);
```

Step-by-Step Explanation

1. **Intent to Enter:** A process sets its flag to true when it wants to enter the critical section.
2. **Turn Assignment:** It sets the turn variable to the other process, giving the other process the chance to enter first if it also wants to.
3. **Waiting Condition:** A process waits if the other process also wants to enter and it is the other's turn.
4. **Critical Section:** Once the condition is false, the process enters the critical section safely.
5. **Exit:** On leaving, the process resets its flag to false, allowing the other process to proceed.

This guarantees:

- **Mutual Exclusion:** Only one process enters CS.
- **Progress:** A process will eventually enter CS if no other is inside.
- **Bounded Waiting;** No process waits indefinitely.

Exercises




When Pi wants to enter the critical section, it sets turn = j. This means Pi gives priority to the other process Pj.



List the three requirements for a correct critical-section solution, and briefly state whether Peterson's algorithm satisfies all three.

The three requirements are:
Mutual exclusion: Only one process can execute inside the critical section at any time.
Progress: If the critical section is idle and at least one process wants to enter, a candidate process can be selected without indefinite delay.
Bounded waiting: A process requesting entry will wait for a limited number of other processes before it is granted access.
Peterson's algorithm satisfies all three requirements for two processes.



```
do {
        flag[i] = true;
        turn = j;
        while (flag[j] && turn == j);
      // Critical Section
        flag[i] = false;
      // Remainder Section

} while (true);
```

Pi and Pj both want to enter the critical section.
Pi runs flag[i]=true; then turn = j
Then Pj runs flag[j]=true; then turn = i
Question:
Which process can enter the critical section? Explain the execution logic.

Part A: True / False

1. Race condition occurs whenever two threads access the same shared variable.
2. Mutual Exclusion means no two threads can run at the same time on the system.
3. Progress requirement prevents all waiting processes from being stuck indefinitely when critical section is free.
4. Bounded-waiting guarantees no starvation for a process requesting entry to critical section.
5. Peterson’s algorithm works correctly if we swap the order of `flag[0]=true;` and `turn=1;`.
6. Both Dekker’s and Peterson’s algorithms use busy-waiting.
7. Peterson’s algorithm can be directly used for 5 concurrent processes.
8. A solution satisfying mutual exclusion is always a correct critical-section solution.
9. Read-only access to shared memory cannot create race condition.
10. In the four-part thread model, Remainder Section contains code that manipulates shared global data.

Part B: Multiple-Choice

1. What is a critical section?
A) A piece of code that only uses private local variables
B) A piece of code that accesses shared resources and may cause race condition
C) The operating system kernel only
D) The spin-waiting loop in entry section

2. The Progress requirement for critical-section solutions ensures that:
A) At most one process runs inside critical section
B) When critical section is free, at least one waiting process can enter
C) A waiting process will enter critical section within 1 ms
D) No process can execute remainder section

3. Bounded-waiting is designed to avoid:
A) Mutual exclusion
B) Race condition
C) Starvation
D) Busy-waiting

4. Which statement about Peterson's algorithm is correct?
A) It uses blocking sleep when waiting
B) It is only valid for two-process scenario
C) It does not need shared variable `turn`
D) It violates mutual exclusion by design

5. Busy-waiting (spin-waiting) means:
A) The waiting process is suspended and moved to block queue
B) The waiting process repeatedly executes a loop consuming CPU time
C) The OS kills waiting processes
D) Context switch never happens

Summary of Section 3.1

01 Race Condition

Occurs when multiple processes concurrently read and write shared data, leading to unpredictable and inconsistent system states.

02 Critical Section

The code segment within a process that accesses shared resources (variables, files, etc.). It must be executed atomically to prevent race conditions.

03 Core Requirements

Three essential rules for a valid solution: **Mutual Exclusion** (only one process inside), **Progress** (no unnecessary blocking), and **Bounded Waiting** (no starvation).

04 Classic Software Solutions

Dekker’ s Algorithm: The first correct solution for two processes, but complex to implement. **Peterson’ s Algorithm:** A simplified, elegant version that is widely used as a teaching example for understanding synchronization logic.

05 Common Weaknesses

Software-only solutions often suffer from **busy waiting** (wasting CPU cycles), and many early algorithms are limited to only two concurrent processes, making them impractical for modern multi-core systems.